DYNAMIC PROGRAMMING

COMPLETE INTERVIEW MASTER SYLLABUS

Java DSA • Foundations to Advanced Patterns

Interview preparation for: Microsoft • Google • Amazon • Meta • Adobe • Qualcomm and other product-based companies

Purpose

A structured checklist of major Dynamic Programming concepts, algorithms, patterns, optimizations and interview problem families. Use it as a learning roadmap—not as a guarantee that every interview question will be covered.

How to use this handbook

1. Understand the problem and identify overlapping subproblems and optimal substructure.
2. Define the DP state precisely: what does dp[i], dp[i][j], or dp[mask][i] represent?
3. Write the recurrence / transition before coding.
4. Set base cases and initialization carefully.
5. Choose a valid evaluation order for tabulation.
6. Implement in Java; test edge cases and explain time and space complexity.
7. Compare recursion + memoization, tabulation and possible space optimization.
8. Practise variations and unfamiliar problems without looking at solutions first.

1. DP Foundations and Core Rules

- Dynamic Programming: definition, when to use it, and recognising DP candidates
- Overlapping subproblems
- Optimal substructure
- Memoization (top-down)
- Tabulation (bottom-up)
- State definition and state dimensions
- State transitions / recurrence relations
- Base cases and initialization
- Transition and iteration order
- Answer extraction from the DP table
- Space optimization and rolling arrays
- State compression
- Recursion tree and complexity analysis
- DP vs Greedy
- DP vs Divide and Conquer
- How to derive a DP solution from brute force recursion

2. 1D DP and Linear-State DP

- Fibonacci Number
- Climbing Stairs
- Min Cost Climbing Stairs
- Frog Jump
- Frog Jump with K distances
- House Robber I
- House Robber II
- Maximum Sum of Non-Adjacent Elements
- Decode Ways
- Maximum Alternating Subsequence Sum
- Count Ways to Reach N
- Maximum Product Subarray (DP formulation)

3. 2D, 3D and Grid DP

- 2D Grid DP fundamentals
- Unique Paths
- Unique Paths II
- Minimum Path Sum

- Maximum Path Sum in Matrix
- Triangle Minimum Path Sum
- Minimum Falling Path Sum
- Cherry Pickup I
- Cherry Pickup II
- Dungeon Game
- 3D DP state transitions
- Multi-dimensional state DP

4. Knapsack, Subset and Partition DP

- 0/1 Knapsack
- Unbounded Knapsack
- Bounded Knapsack
- Subset Sum
- Equal Sum Partition
- Minimum Subset Sum Difference
- Count Subsets with Given Sum
- Target Sum
- Perfect Sum Problem
- Coin Change: minimum number of coins
- Coin Change II: number of ways
- Rod Cutting
- Partition with Given Difference
- Count Partitions with Given Difference

5. Subsequences and Sequence DP

- Longest Common Subsequence (LCS)
- Longest Common Substring
- Longest Increasing Subsequence (LIS)
- LIS with binary-search optimisation
- Number of Longest Increasing Subsequences
- Longest Bitonic Subsequence
- Largest Divisible Subset
- Printing / reconstructing an LCS
- Shortest Common Supersequence
- Minimum Insertions and Deletions to Transform a String
- Distinct Subsequences

- Longest Repeating Subsequence

6. String DP

- Edit Distance (Levenshtein distance)
- Wildcard Pattern Matching
- Regular Expression Matching
- Longest Palindromic Subsequence
- Longest Palindromic Substring (DP approach)
- Palindrome Partitioning
- Minimum cuts for palindrome partitioning
- Count Palindromic Subsequences
- Interleaving String
- Word Break
- Word Break II
- Scramble String
- Minimum Insertions to Make a String Palindrome
- Distinct Subsequences

7. Stock / State-Machine DP

- Best Time to Buy and Sell Stock I
- Best Time to Buy and Sell Stock II
- Best Time to Buy and Sell Stock III
- Best Time to Buy and Sell Stock IV
- Stock with Cooldown
- Stock with Transaction Fee
- State-machine DP for buy / hold / sell states
- Transaction-limit DP

8. Interval / Range DP

- Matrix Chain Multiplication
- Burst Balloons
- Minimum Cost to Cut a Stick
- Boolean Parenthesization
- Palindrome Partitioning II
- Strange Printer
- Merge Stones
- Optimal Binary Search Tree

- Minimum Score Triangulation of Polygon
- Scramble String (interval formulation)

9. Tree DP

- Tree Diameter using DP
- Maximum Path Sum in Binary Tree
- House Robber III
- Maximum Independent Set on a Tree
- Subtree DP
- Rerooting DP
- Tree Matching DP
- Multi-state Tree DP
- Counting subtrees with specified properties

10. Graph DP and Related Relaxation Algorithms

- Shortest path in a DAG using topological order
- Longest path in a DAG
- Counting paths in a DAG
- General DP on Directed Acyclic Graphs
- Floyd–Warshall (all-pairs shortest paths; DP-style recurrence)
- Bellman–Ford (edge-relaxation shortest paths; related recurrence technique)
- Hamiltonian Path DP
- Travelling Salesperson Problem (TSP) with bitmask DP
- Bitmask DP on graphs

11. Bitmask, Subset and Profile DP

- Bitmask representation and subset iteration
- Subset DP
- Assignment Problem
- Minimum-Cost Assignment
- Travelling Salesperson DP
- Hamiltonian Path counting
- SOS DP (Sum Over Subsets)
- Profile DP
- Broken Profile DP

12. Digit DP

- Digit DP fundamentals
- Tight / loose state
- Counting numbers with a given digit sum
- Range Digit DP
- Counting numbers with no repeated digits
- Counting numbers containing required digits
- Digit DP with remainder modulo K
- Digit DP with multiple state constraints

13. Combinatorial, Probability and Game DP

- Binomial Coefficient DP
- Pascal's Triangle
- Catalan Number DP
- Counting combinations
- Counting permutations
- Expected Value DP
- Probability DP
- DP on game states

14. Advanced DP Optimisation

- Divide and Conquer DP Optimisation
- Convex Hull Trick
- Knuth Optimisation
- Monotonic Queue Optimisation
- DP with Segment Tree
- DP with Fenwick Tree / Binary Indexed Tree
- Coordinate Compression + DP
- Meet in the Middle
- SOS DP optimisation
- Rerooting optimisation
- Profile DP optimisation
- Persistent DP (advanced concept)

15. Cross-Cutting DP Techniques and Reconstruction

- Take / not-take transitions
- Counting ways vs minimising cost vs maximising score
- Boolean / feasibility DP

- DP with multiple states
- DP with limited transactions / resources
- Reconstructing an optimal path or selected items
- Parent / choice arrays for solution reconstruction
- Rolling-array and in-place DP safety
- Modulo arithmetic in counting DP
- Handling impossible states and infinity sentinels
- Overflow-safe Java types and bounds
- Memoization key design and avoiding state collisions

16. Mandatory Algorithm Write-up Format

For every algorithm or problem family, prepare the following. This makes the syllabus actionable rather than a list of names.

1. Definition and use case: what it solves and when it applies.
2. Rule / core idea: the principle and why DP is appropriate.
3. State definition: exact meaning of every DP index and dimension.
4. Recurrence: mathematical transition, including all choices.
5. Base cases: empty, smallest and impossible states.
6. Initialization: initial values and sentinel values.
7. Evaluation order: recursion order or tabulation loop order.
8. Algorithm steps: clear pseudocode or numbered steps.
9. Java implementation: readable code with correct data types.
10. Dry run: a small example showing state changes.
11. Complexity: time, DP storage and auxiliary space.
12. Optimisation: memoization, tabulation and space optimisation where valid.
13. Reconstruction: how to return the actual path / subset / sequence when asked.
14. Variations: common twists and edge cases.
15. Practice: representative Easy, Medium and Hard problems.

17. Interview Readiness Checklist

- Can identify whether a problem is DP, greedy, graph, or another pattern.
- Can start with brute-force recursion and derive overlapping states.
- Can define a correct state without relying on memorised code.
- Can derive and explain the recurrence.
- Can select correct base cases and iteration order.
- Can convert memoization to tabulation.
- Can optimise space without breaking dependencies.
- Can reconstruct an optimal solution when required.
- Can analyse time and space complexity.
- Can implement in Java and test boundary cases.
- Can solve mixed, unseen problems under time limits and explain trade-offs.

18. Suggested Study Order

1. Foundations → 1D DP → grid / 2D DP
2. Knapsack → subset / partition → subsequence DP
3. String DP → stock / state-machine DP
4. Interval DP → tree DP → DAG / graph DP
5. Bitmask / profile DP → digit DP
6. Combinatorial / probability / game DP

7. Advanced optimisation → mixed timed practice and mock interviews

Important: This is a broad interview-oriented syllabus, not a promise that every company or role will ask every topic. Prioritise core patterns and practise deriving solutions; advanced topics are role- and company-dependent.